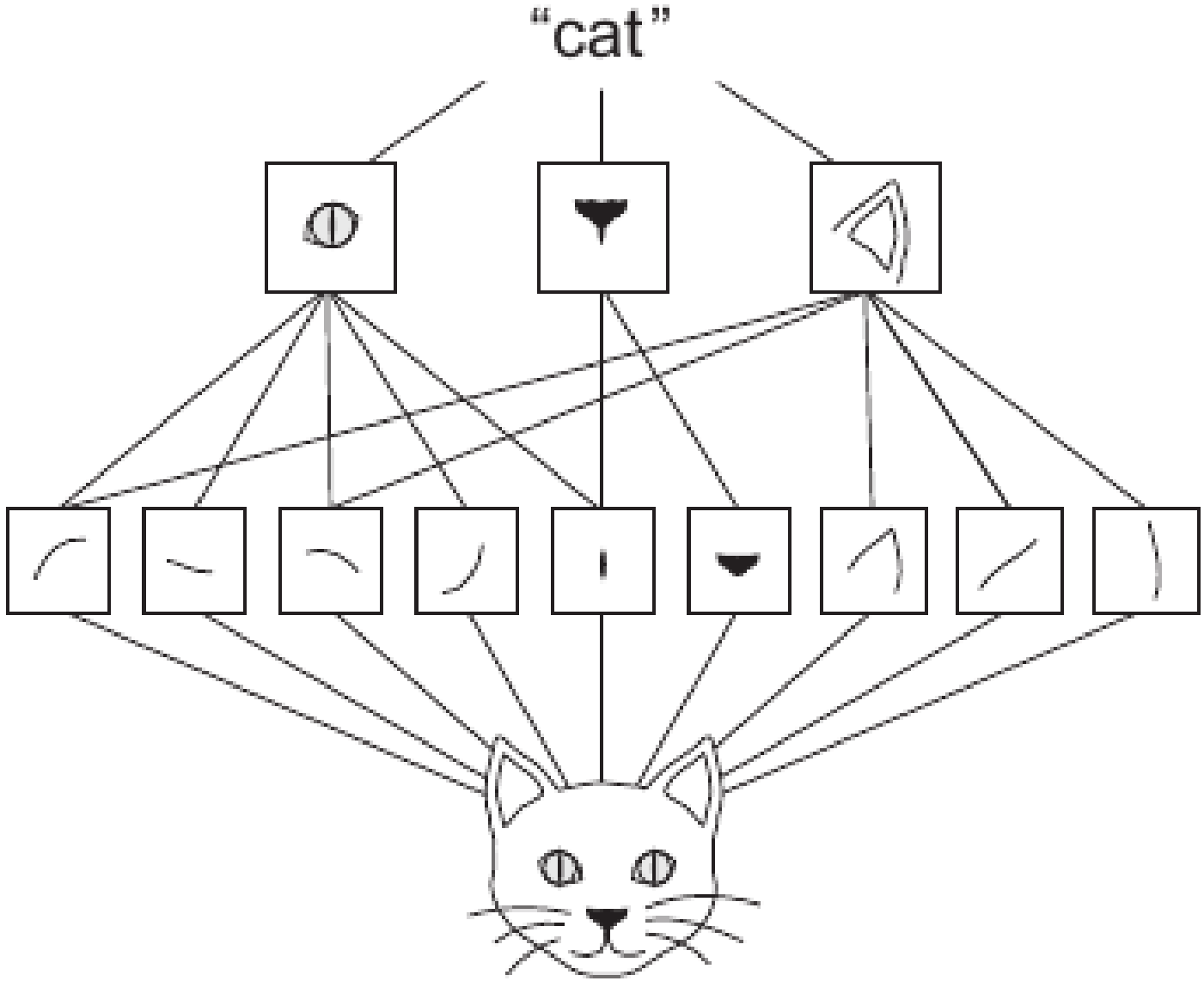
Convolutional neural network - capitolul 14 Geron

Where do we use CNN?

- Finding patterns in images: to recognize objects, faces, and scenes.
- They can also be quite effective for classifying non-image data such as audio, time series, and signal data.
- self-driving vehicles, face-recognition applications, medical images

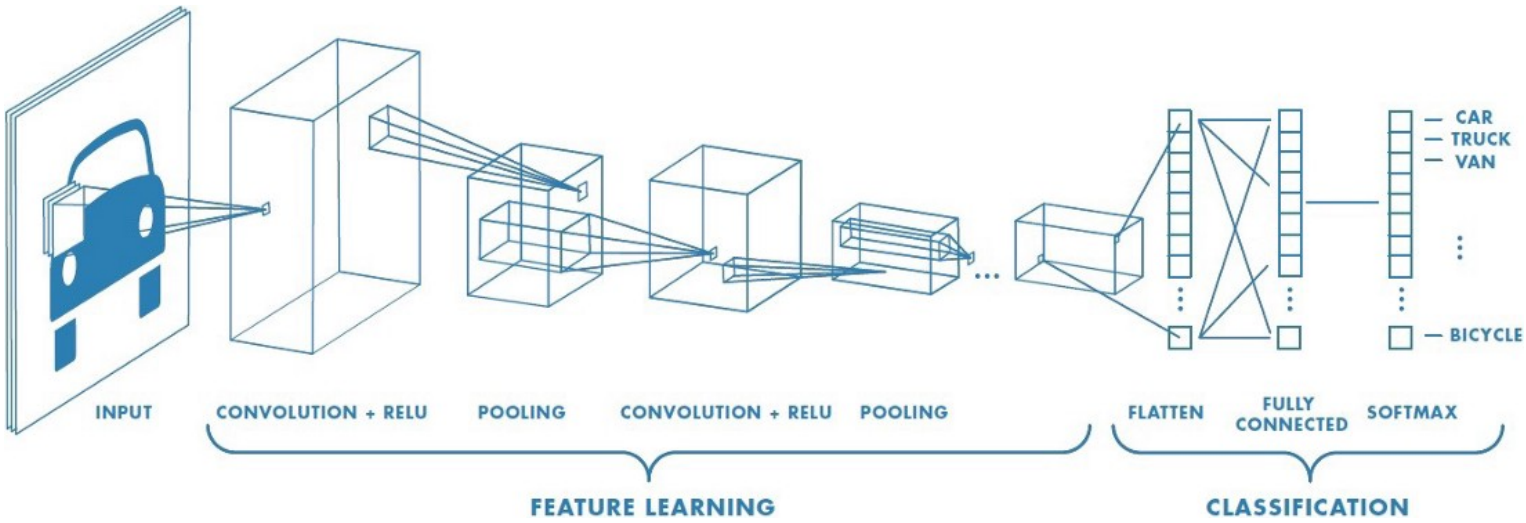
CNN - Hierachies of patterns



1

[1] Chollet. Deep Learning with python

CNN - how?


CAR
TRUCK
VAN
BICYCLE
INPUT
CONVOLUTION + RELU
POOLING
CONVOLUTION + RELU
POOLING
FLATTEN
FULLY CONNECTED
SOFTMAX
FEATURE LEARNING
CLASSIFICATION

CNN - how?

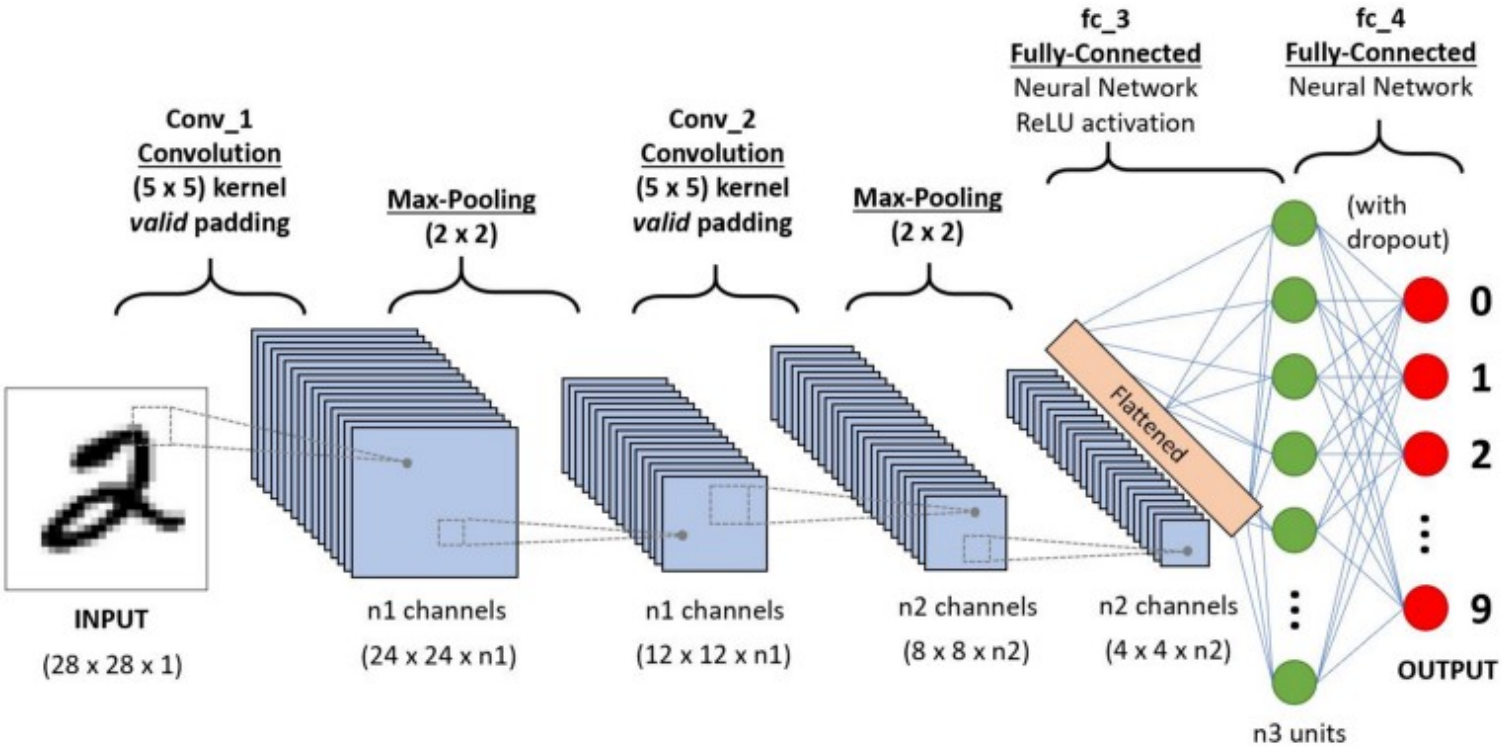
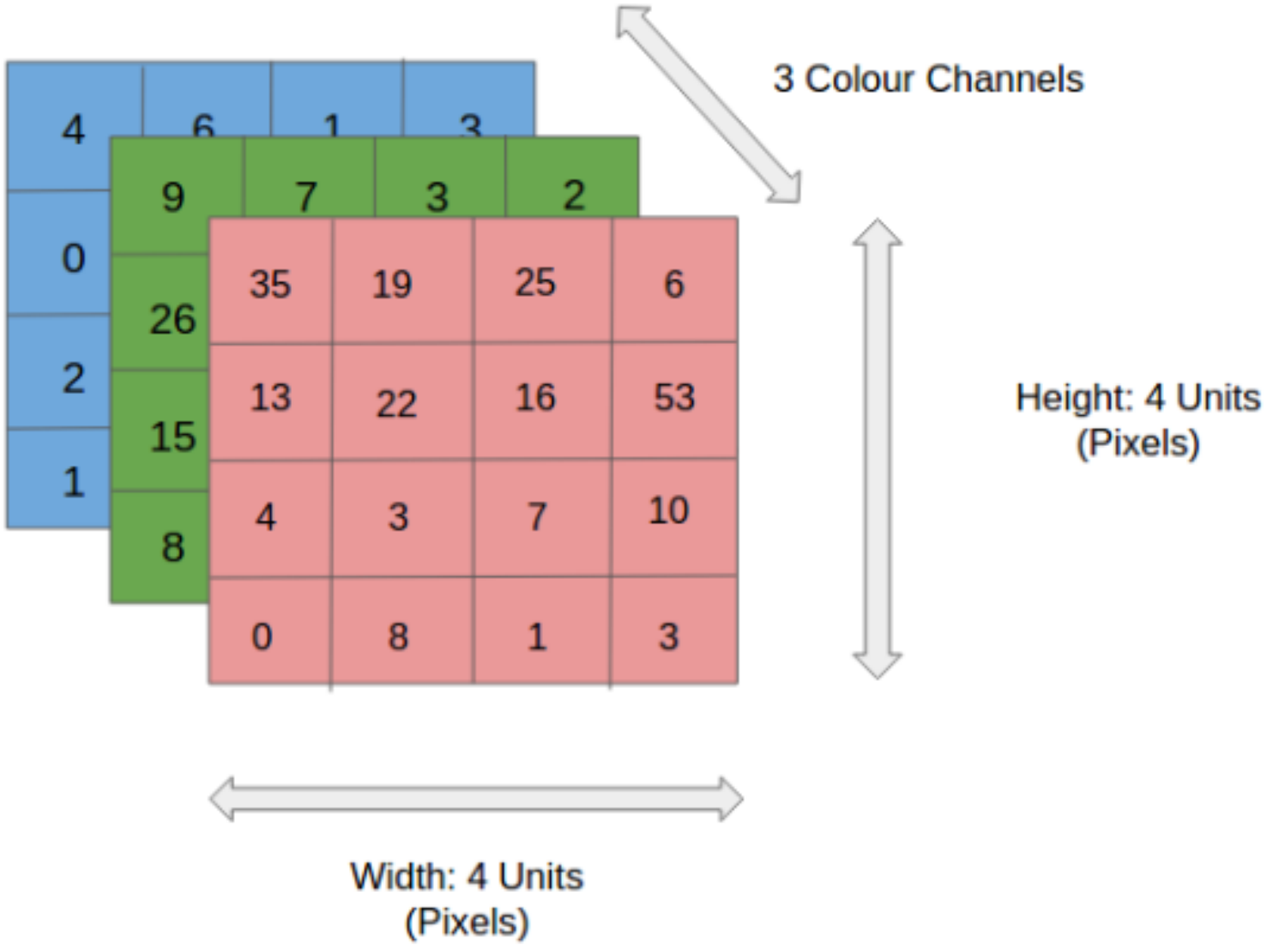

fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
(with
dropout)
Flattened
0
1
2
9
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
n3 units
OUTPUT

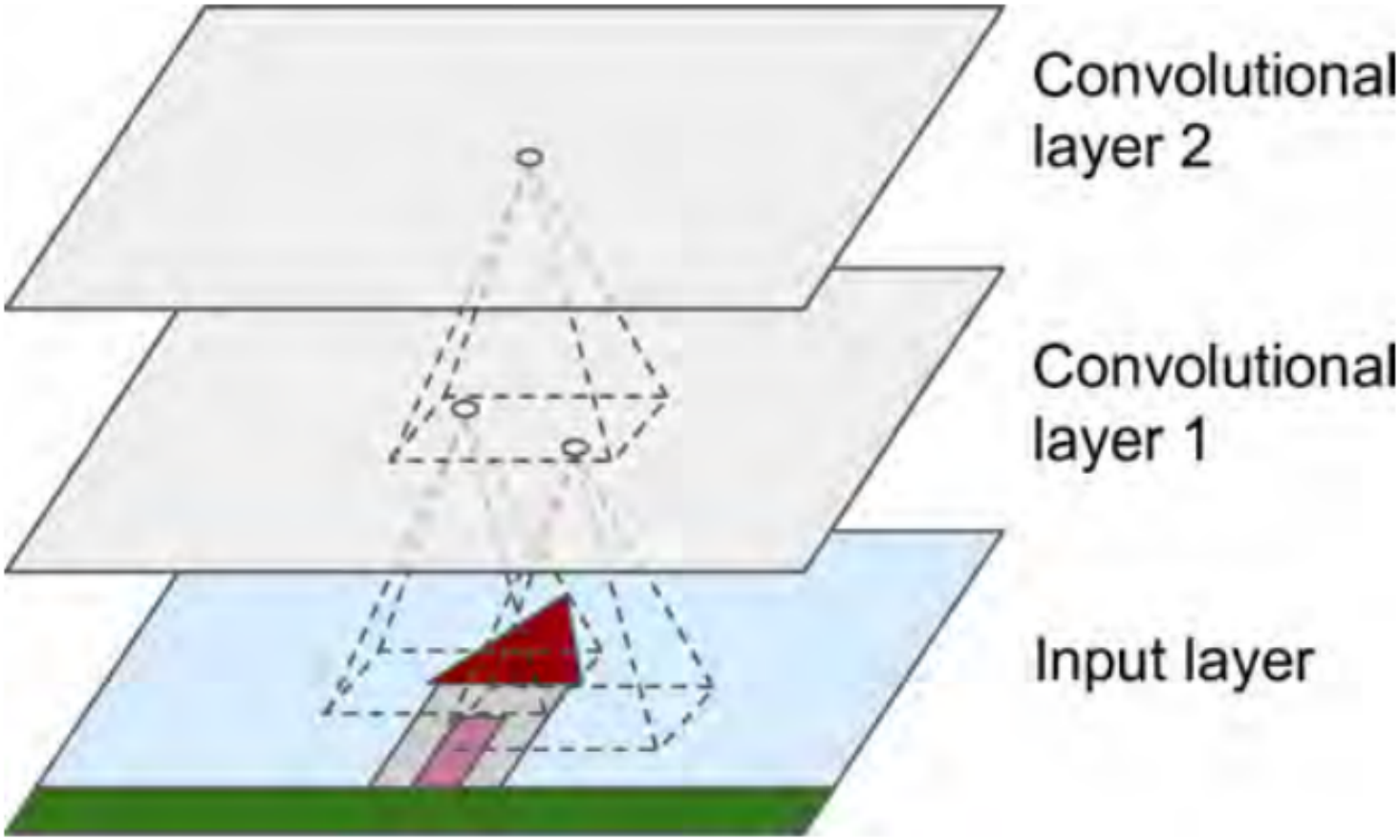
Image RGB - Grayscale



CNN

- Receptive fields
  - First convolutional layer: neurons are not connected to every single pixel in the input image, but only pixels in their receptive fields
  - Second convolutional layer: each neuron is connected only to neurons located within a small rectangle
- the networks concentrates on
  - low-level features in the first hidden layer, then
  - assemble them into higher-level features in the next hidden layer,
  - and so on.
- reduce the images into a form which is **easier** to process, **without losing features** which are critical for getting a good prediction

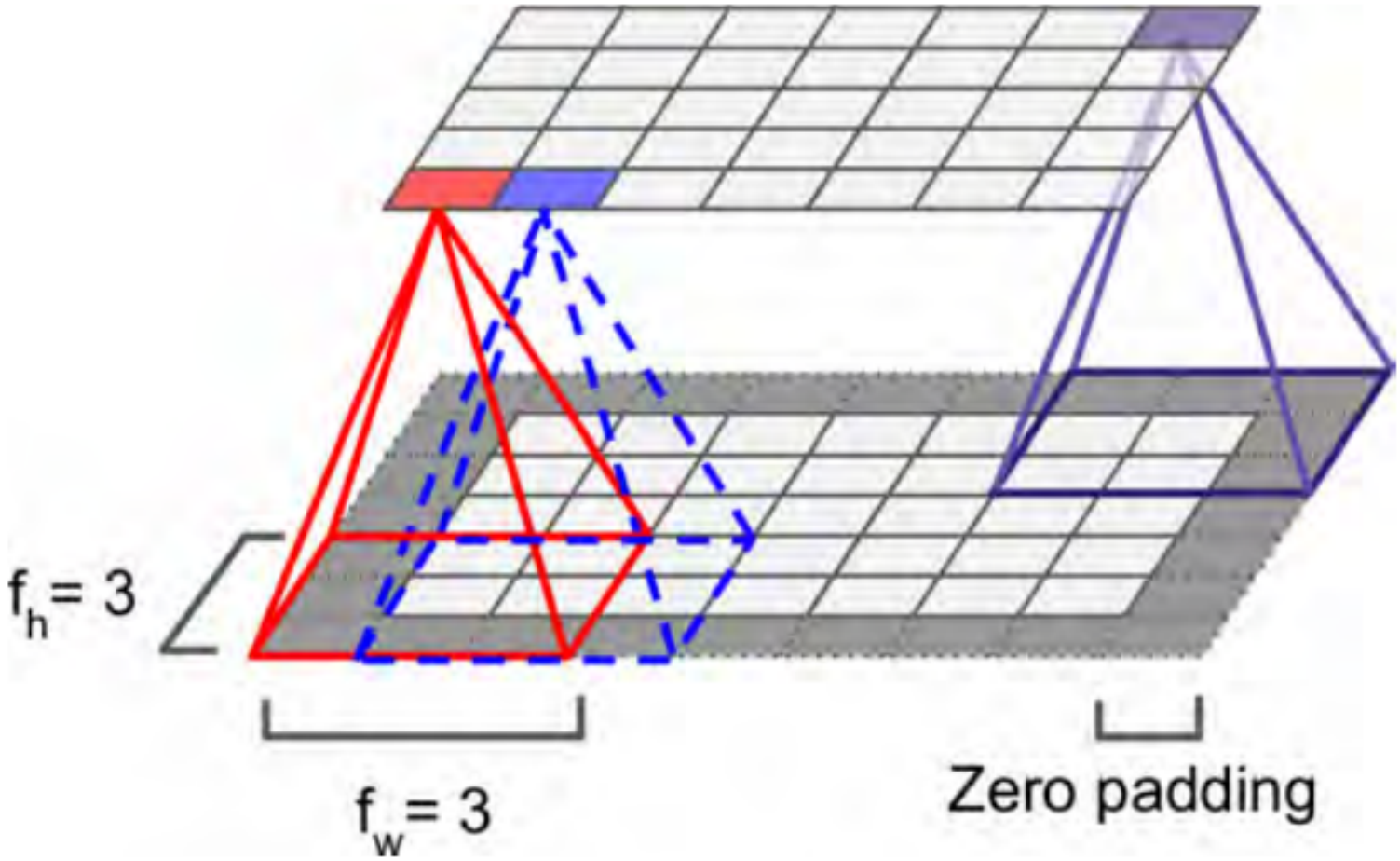
CNN layers with rectangular local receptive fields



2

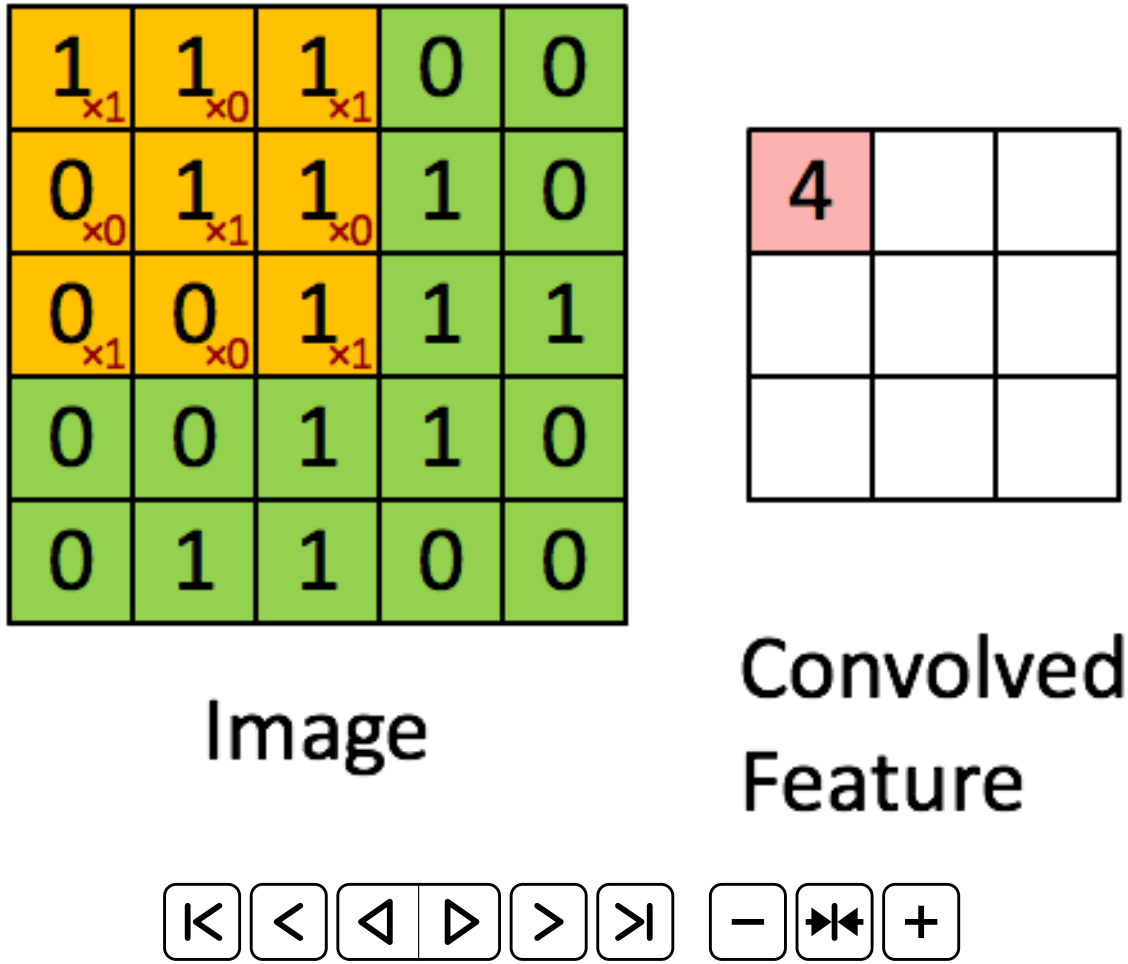
[2] Pictures taken from Geron. Hands-on Machine Learning

Connections between layers (with zero padding)

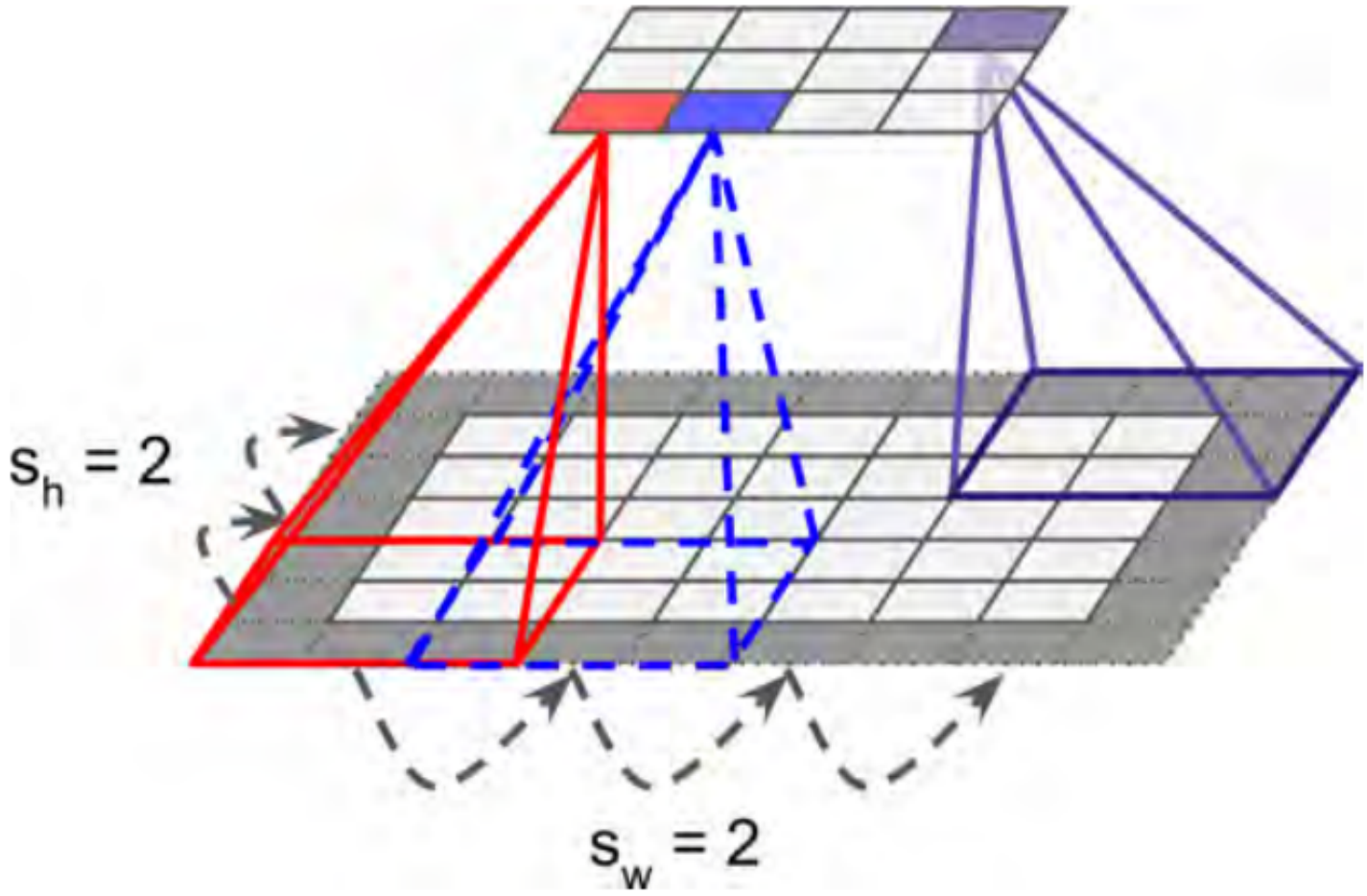


- neuron i, j - connected to neurons in previous layer in rows i to $i + f_h - 1$, columns j to $j + f_w - 1$.
- the layer has the same height and width as the previous layer (with zero padding)

Convolution

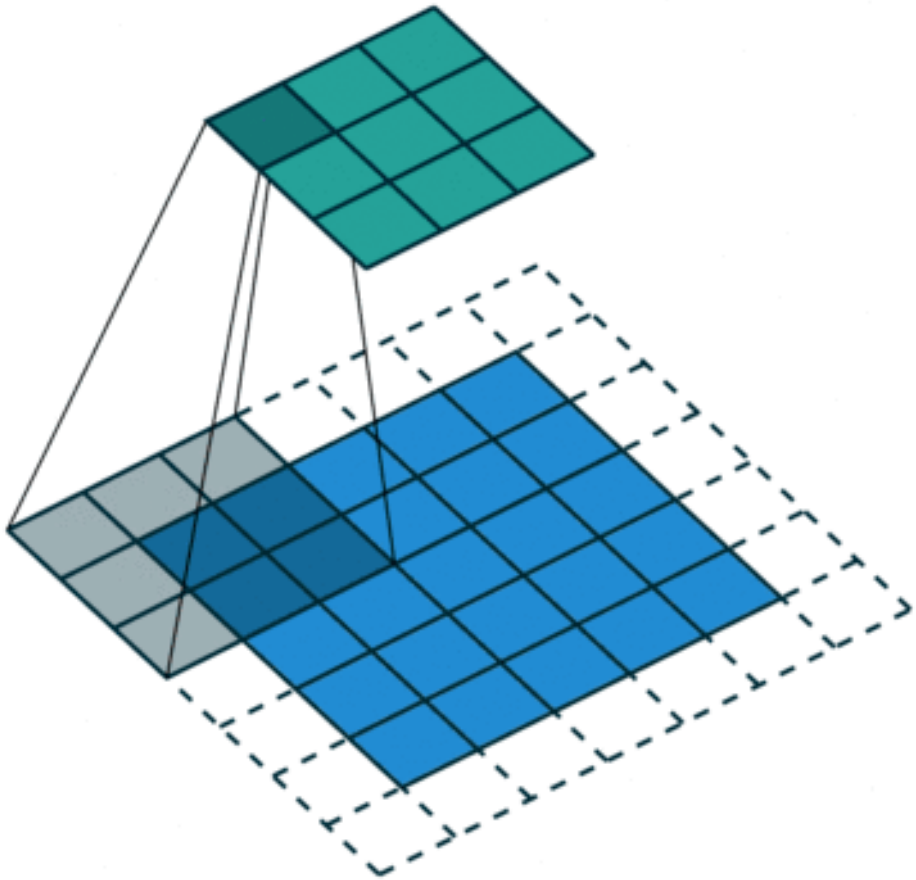


Reduce dimensionality using a stride

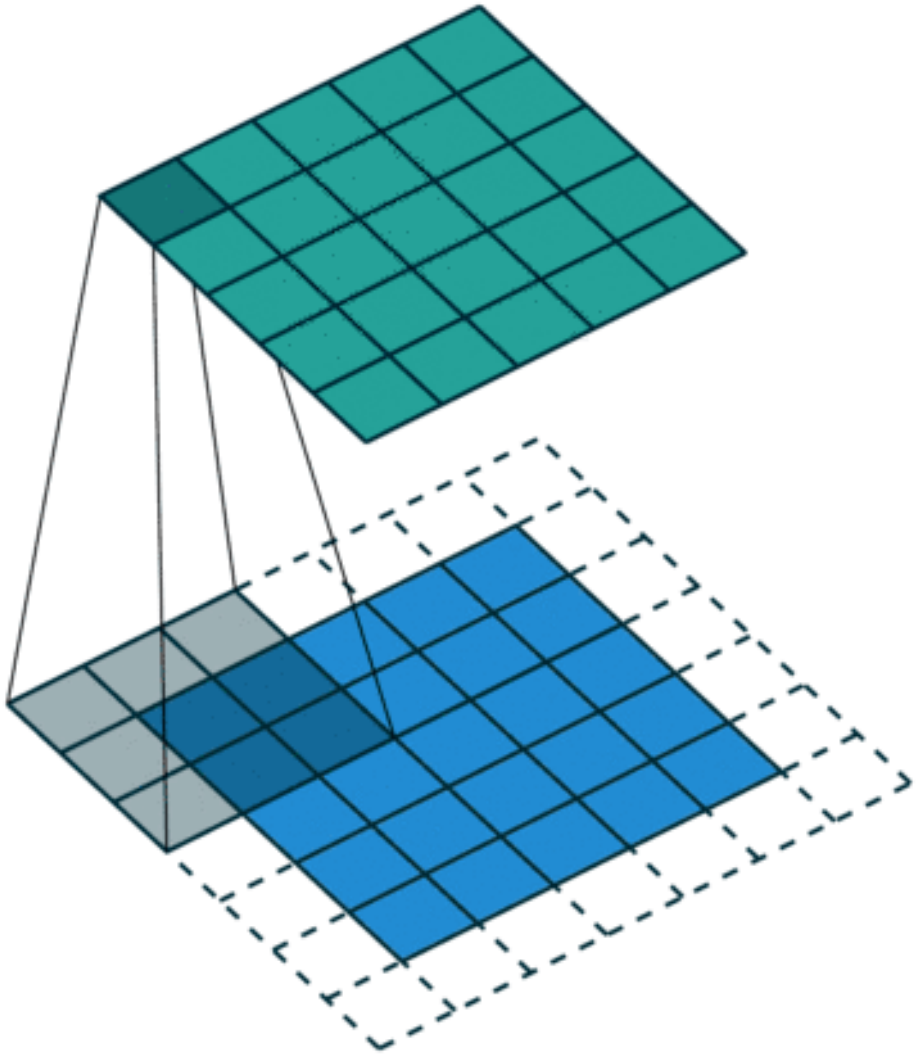


Input layer : $5x7$. Filter: $3x3$. Stride: 2.Result: $3x4$

Convolution with stride length=2

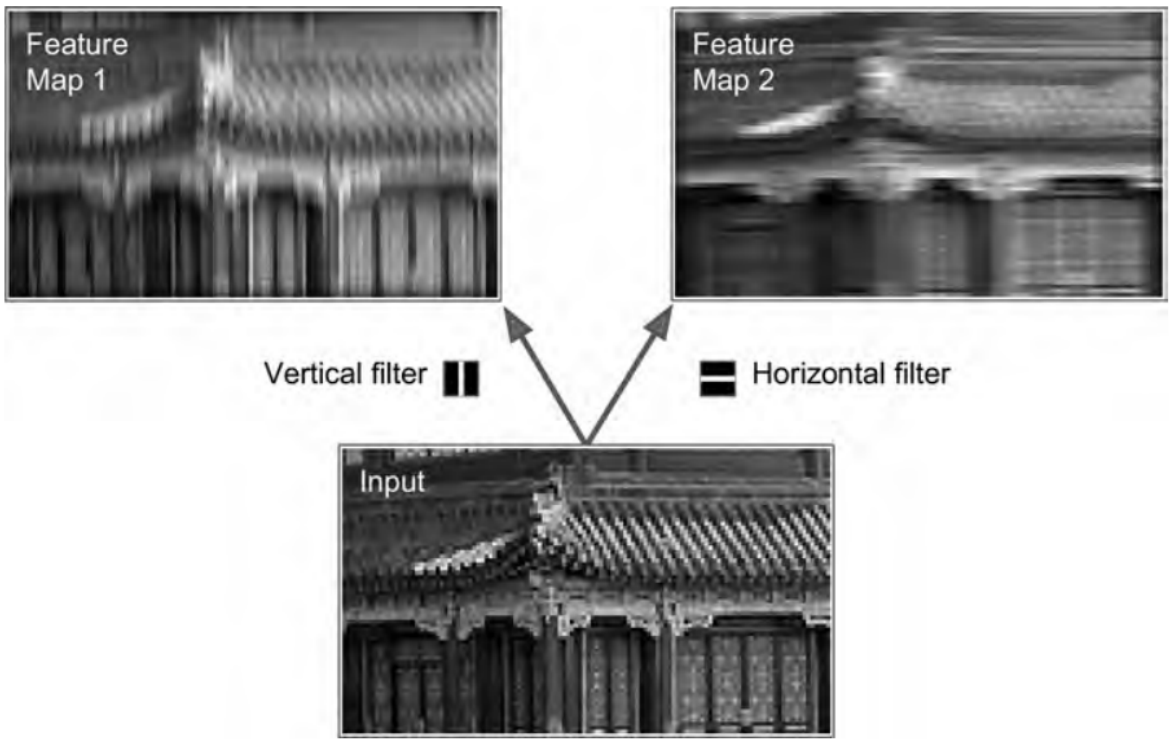


Padding: 5x5x1 image is padded with 0s to create a 5x5x1 image

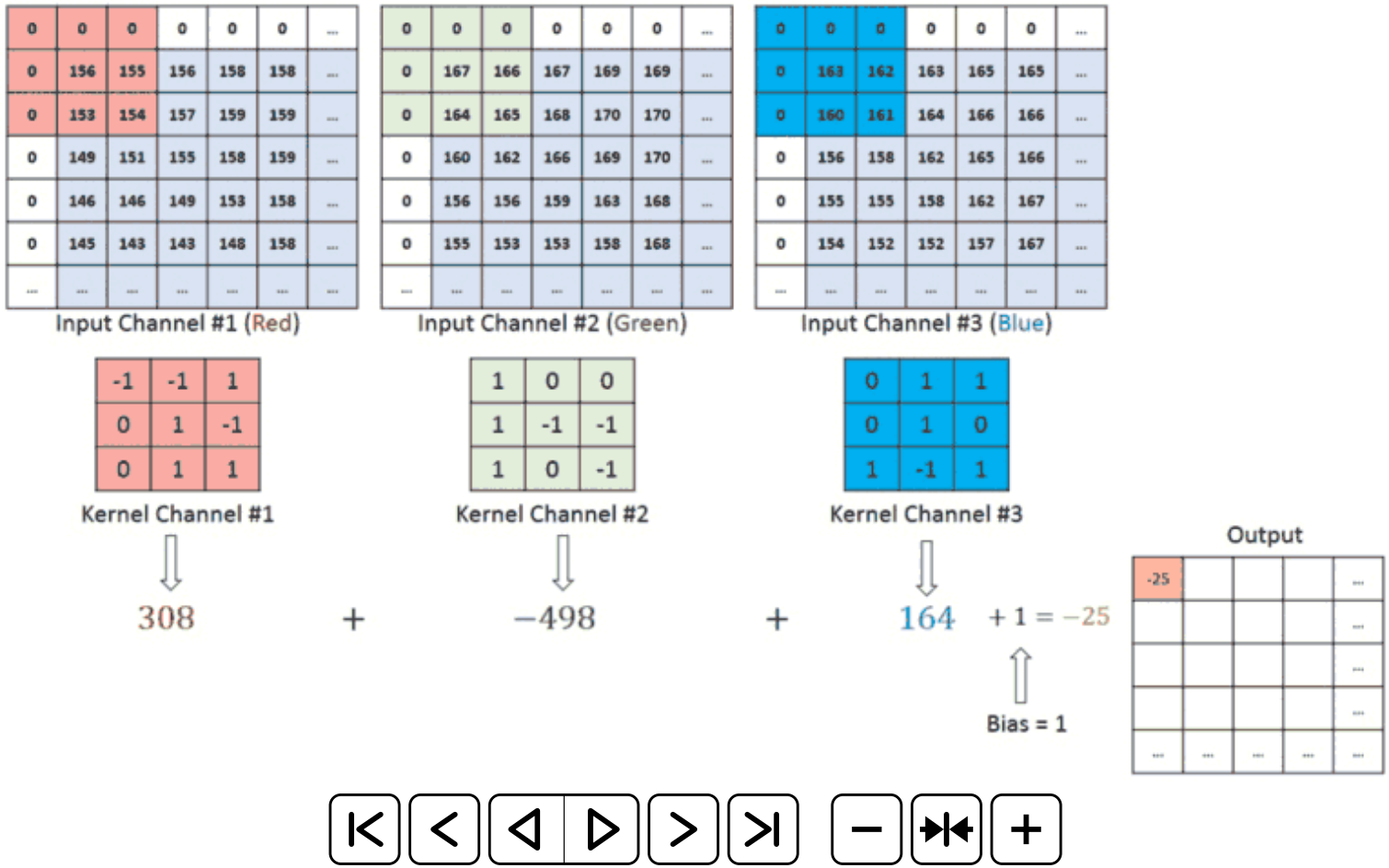


Filters (kernels)

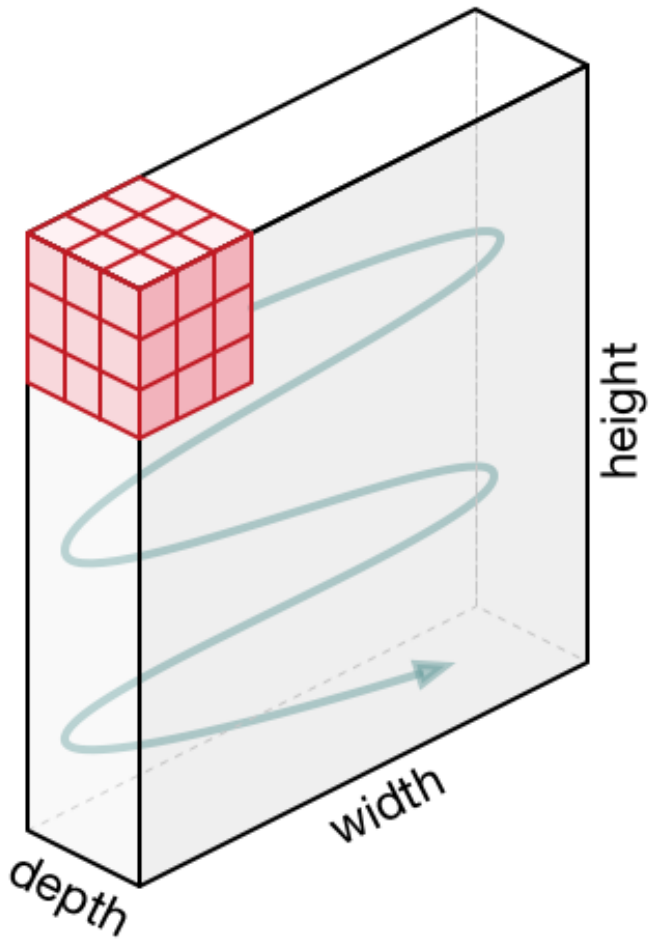
- a neuron's **weight** can be seen as a small image the size of the receptive field
- a layer full of neurons using the same filter gives a feature map = highlights the areas in an image that are most similar to the filter



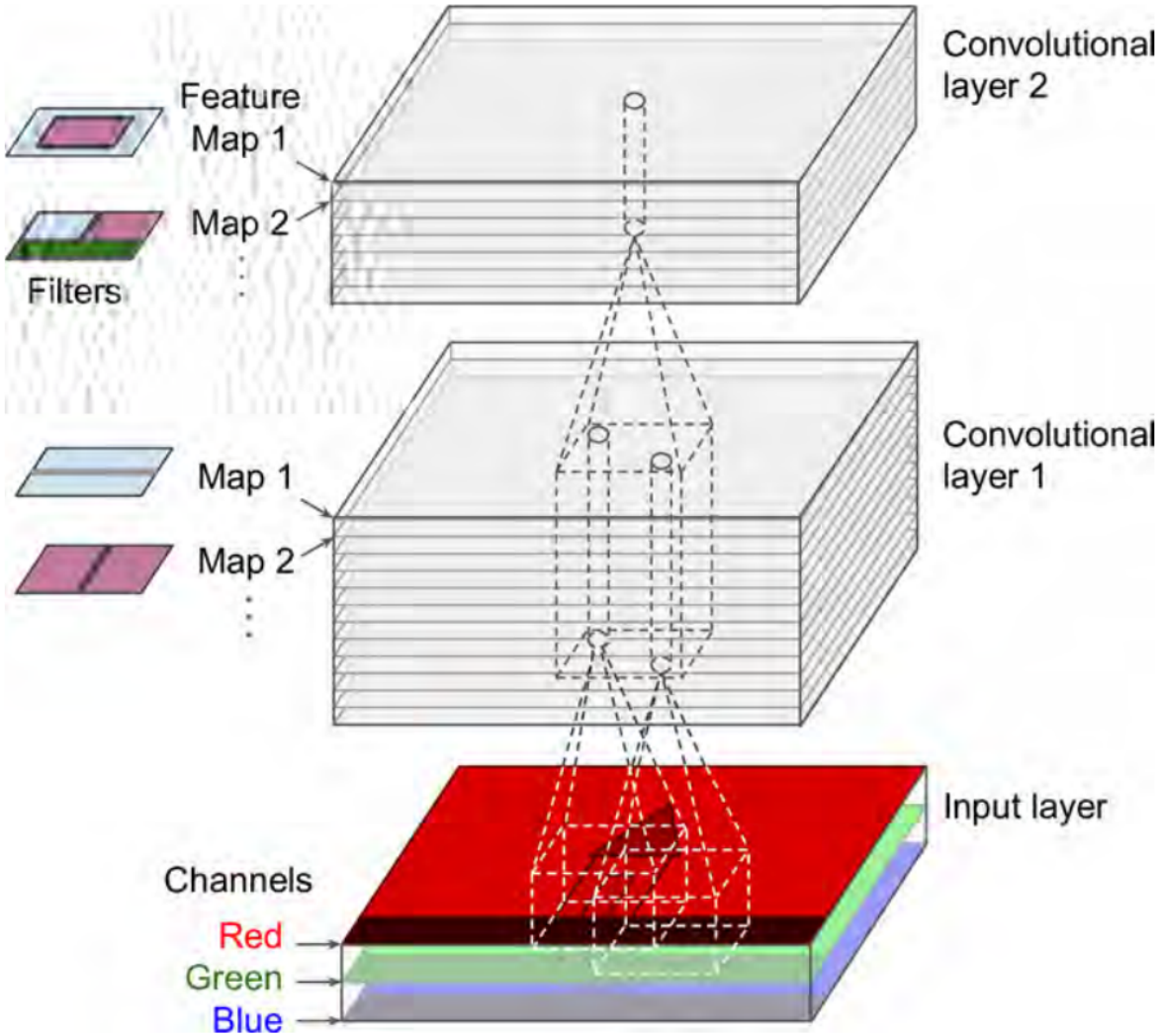
Convolution - 3 canale intrare



Kernel K - moving



Convolution layers with multiple feature maps and image with three channels



Stacking multiple feature maps

Obs: All the previous representations were simplifications

- a convolutional layer is composed of several feature maps of equal sizes
- within one feature map, all neurons share the same parameters
- different feature maps may have different parameters
- a neuron's receptive field includes a limited number of neurons but across all the feature maps of the previous layer

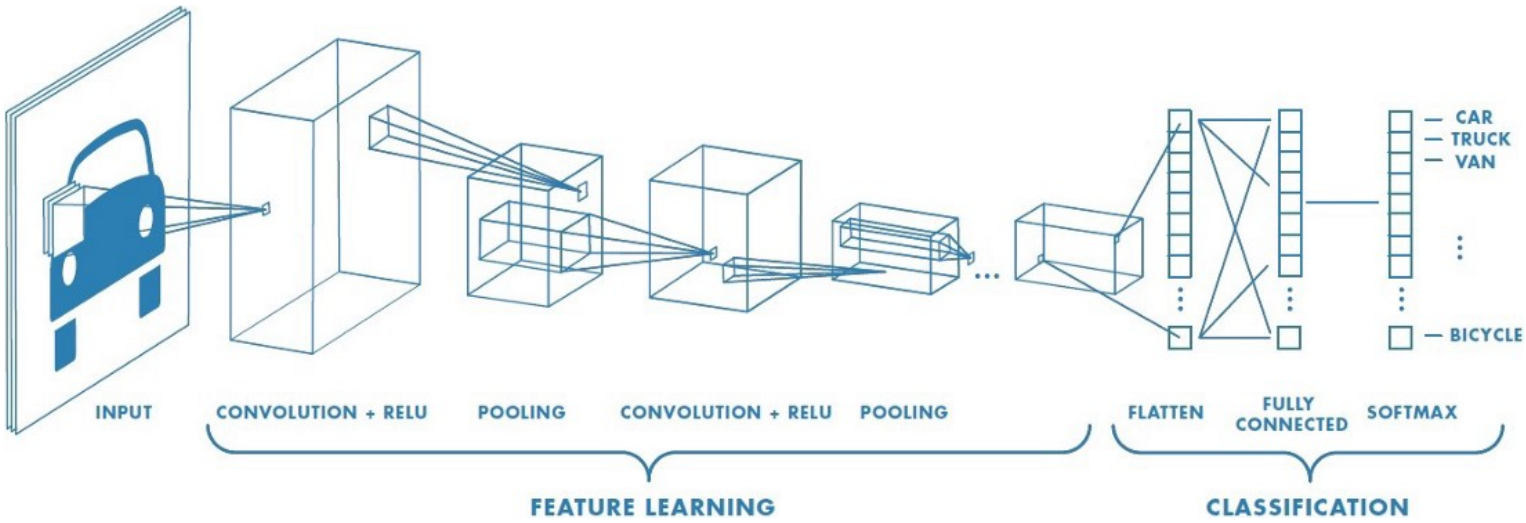
cont. Feature maps

A neuron located

- in row i, column j of the feature map k is connected
- to the outputs of the neurons in the previous layer located in rows ixs_w to $ixs_w + f_w - 1$, columns jxs_h to $jxs_h + f_h - 1$ across all feature maps

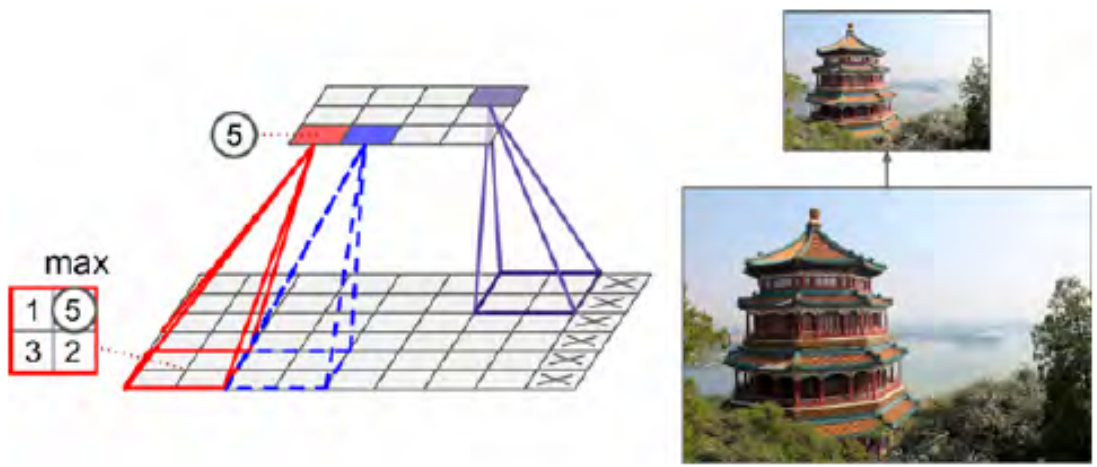
All neurons located in the same row and column but in different feature maps are connected to the outputs of the exact same neurons in the previous layer

CNN - convolution + ?



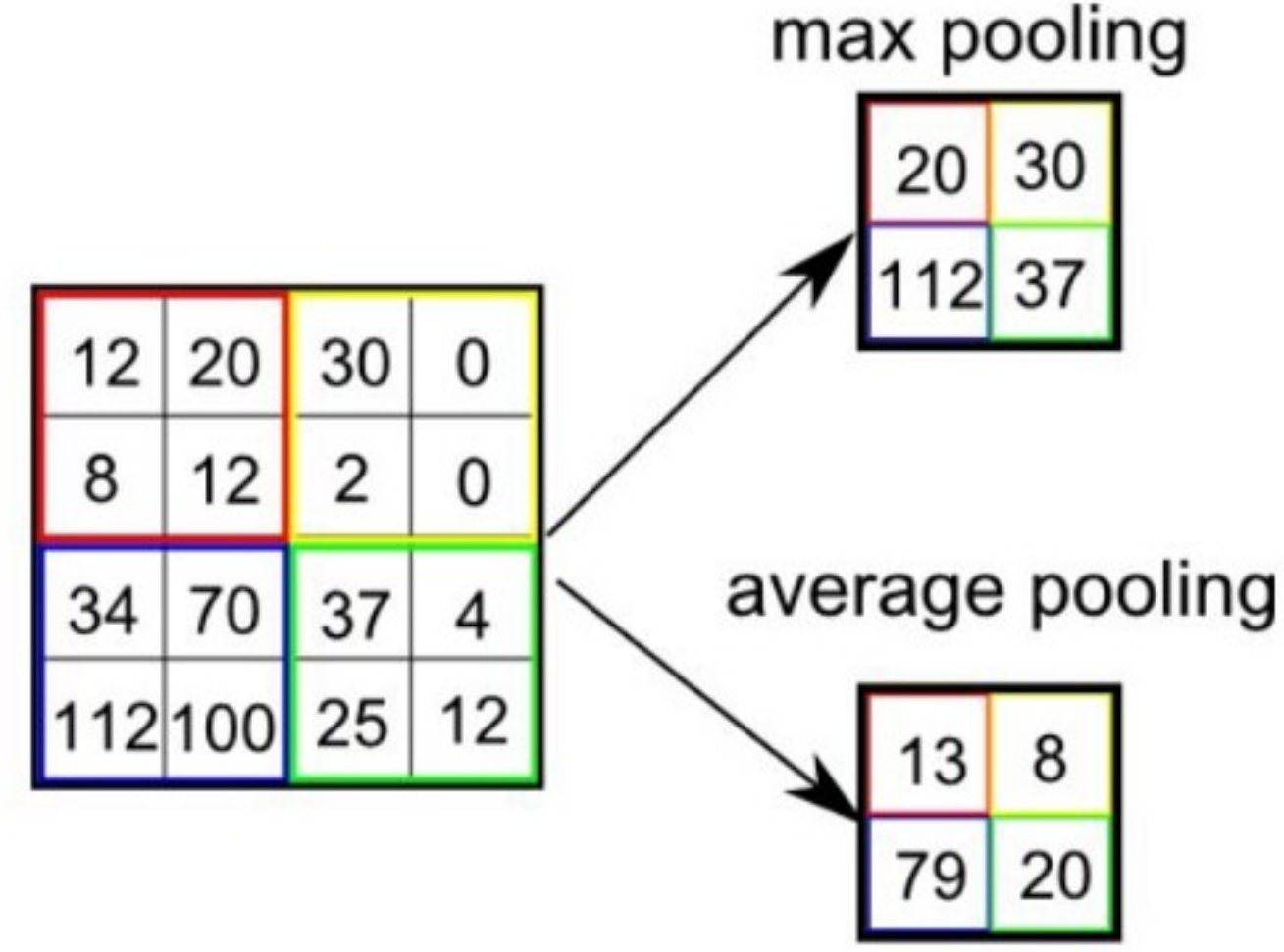
Pooling layer: max, mean

- Subsample the input image - reduce number of parameteres - reduce overfitting
- Required info: size, stride, padding type
- Has no weights
- Works on every input channel independently: output depth is the same as input depth



max pooling, 2x2, stride 2, no padding

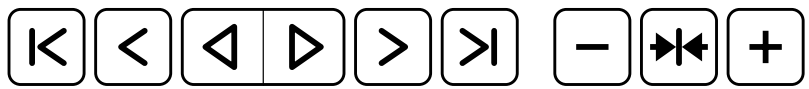
Pooling


max pooling
20 30
112 37
12 20 30 0
8 12 2 0
34 70 37 4
112 100 25 12
average pooling
13 8
79 20

Pooling cu Valid padding

1	2	3
4	5	6
7	8	9
10	11	12

padding	VALID
filter	2x2
stride	2x2
input	4x3
output	2x1

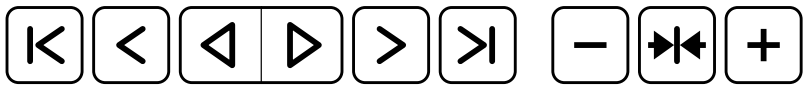


With "VALID" padding, there's no "made-up" padding inputs. The layer only uses valid input data.

Pooling cu SAME padding

1	2	3
4	5	6
7	8	9
10	11	12

padding	SAME
filter	2x2
stride	2x2
input	4x3
output	2x2

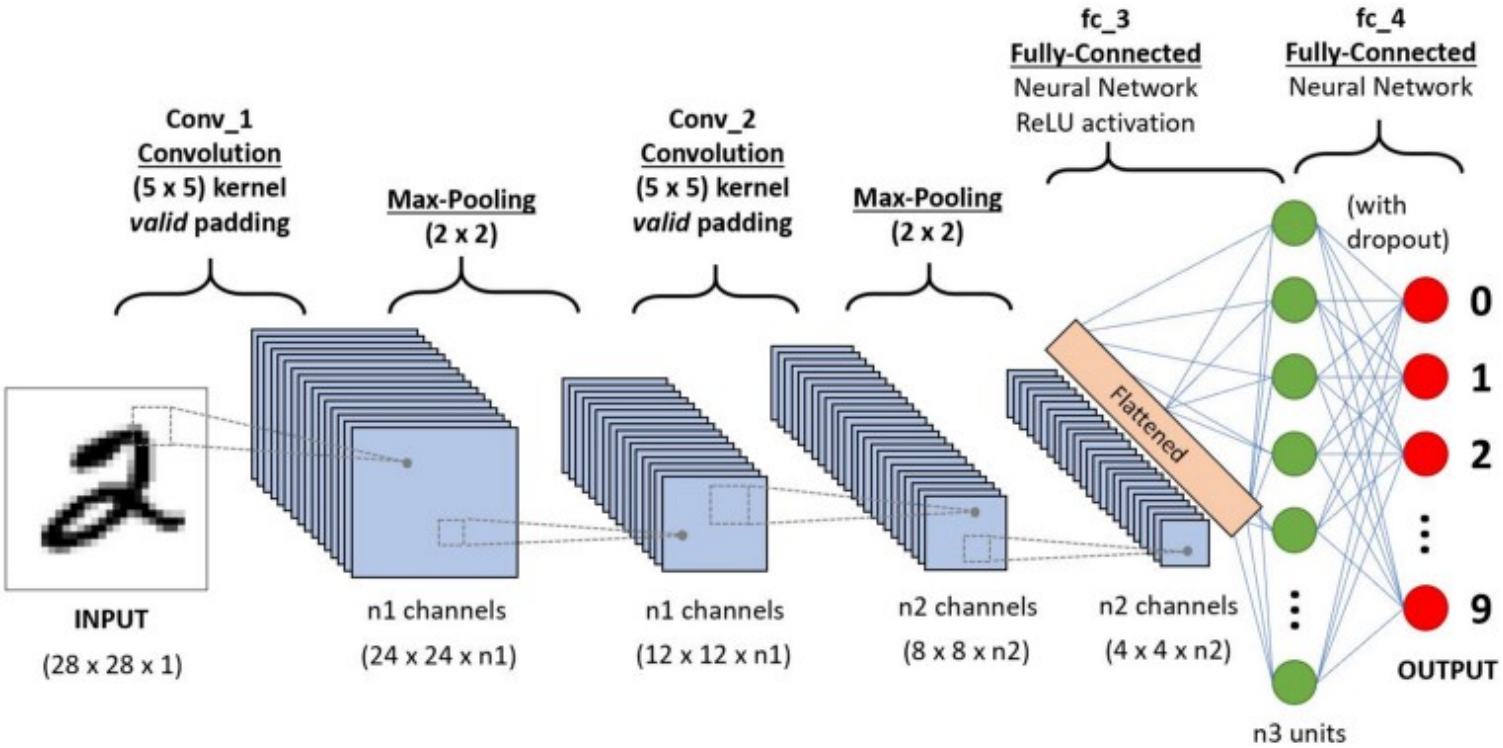


SAME - it uses zero padding if necessary
With "SAME" padding, **if you use a stride of 1**, the layer's outputs will have the same spatial dimensions as its inputs.

Why not simply use fully connected layers

- #parameters for dense layer: input image 100x100. First dense layer 1000 neurons (Obs: it restricts the amount of information transmitted to the next layer) $\rightarrow 10^7$ parameters for only one layer!!!
- CNN: the learnt patterns are translation invariants: a certain pattern can be recognized anywhere
- CNN can learn hierarchies of patterns

Fully connnected



nice visualisation

Observations

A. Geron: A common mistake is to use convolution kernels that are too large. You can often get the same effect as a $9x9$ kernel by stacking two $3x3$ kernels on top of each other, for a lot less compute.

Observations

- CNNs eliminate the need for manual feature extraction—the features are learned directly by the CNN.
- CNNs produce highly accurate recognition results.
- CNNs can be retrained for new recognition tasks, enabling you to build on pre-existing networks.

Next time: transfer learning, efficient architectures

Run in tensorflow

- notebook CNN:sursa: CNN notebook
- Image classification and augmentation TensorflowTutorials
- Transfer learning Transfer learning and fine tuning